«ОСНОВЫ БАЗ ДАННЫХ И ЯЗЫК SQL НА ПРИМЕРЕ SQLITE»

Цели работы и ожидаемые результаты

Познакомиться с основами баз данных и языком SQL на практике

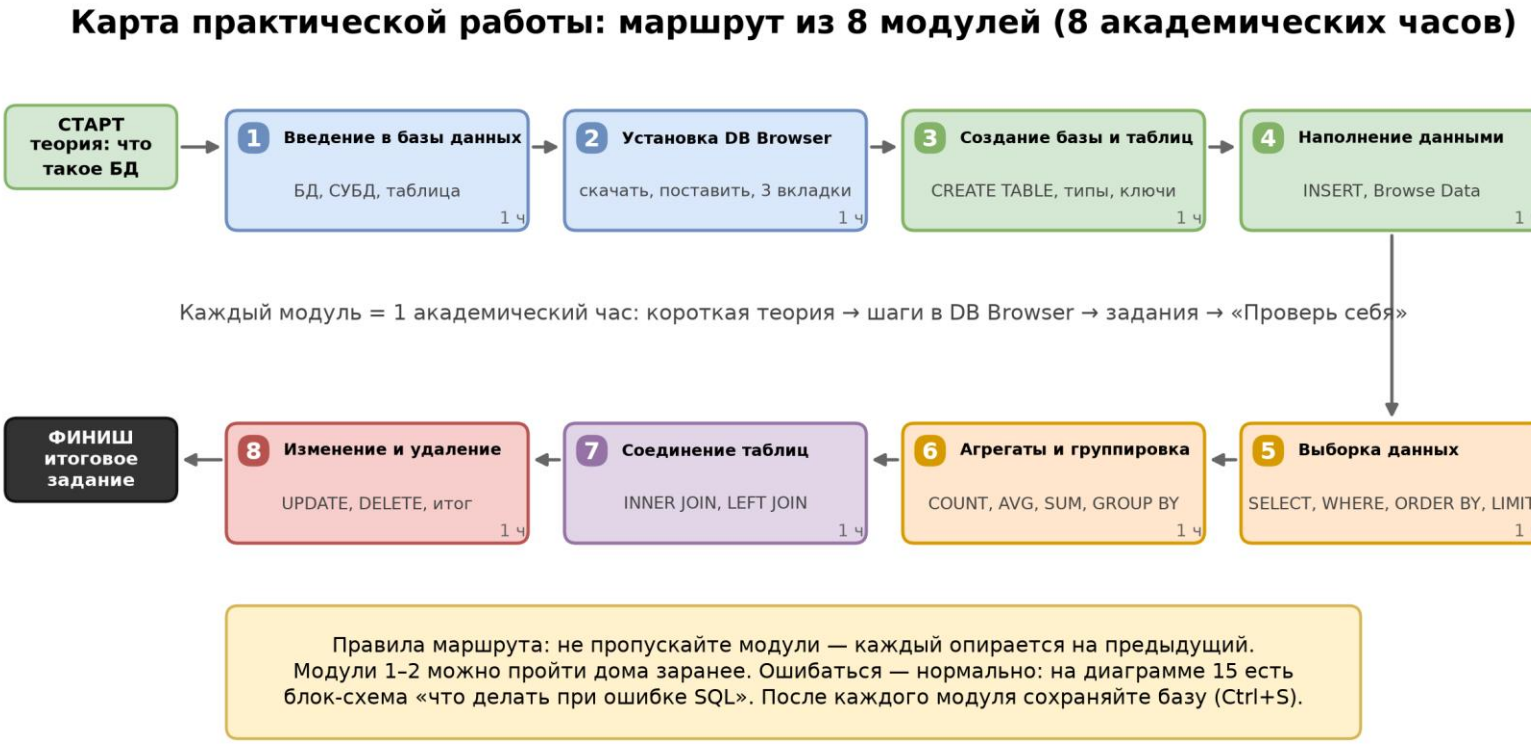
Порядок оценивания

Работа оценивается по накопительной системе, максимум — 100 баллов:

Составляющая	Баллы	За что начисляются
Модули 1–8 (по 8 баллов за каждый)	64	Выполнены шаги модуля, даны ответы «Проверь себя», сдано задание модуля
Итоговое самостоятельное задание	20	6 запросов выполнены верно, результаты сохранены (см. критерии в конце)
Контрольные вопросы	10	Не менее 8 правильных ответов из 12 устных/письменных вопросов
Аккуратность и самостоятельность	6	База названа правильно, ошибки исправлялись самостоятельно, данные не испорчены

Сумма баллов	Оценка
90–100	отлично (5)
75–89	хорошо (4)
60–74	удовлетворительно (3)
менее 60	неудовлетворительно (2), работа сдаётся повторно

Карта работы: маршрут из 8 модулей



Правила маршрута: не пропускайте модули — каждый опирается на предыдущий. Модули 1–2 можно пройти дома заранее. Ошибаться — нормально: на диаграмме 15 есть блок-схема «что делать при ошибке SQL». После каждого модуля сохраняйте базу (Ctrl+S).

Модуль 1. Введение в базы данных

Что такое база данных и зачем нужна СУБД

Подход 1. Данные просто в файлах

Файл «Студенты.xlsx» (или блокнот)

↓

Вторая копия — «Студенты_ФИНАЛ.xlsx»

- ✗ Поиск и сортировка — вручную
- ✗ Данные дублируются
- ✗ Копии расходятся, устаревают
- ✗ Двоим трудно работать сразу
- ✗ Нет защиты от ошибок ввода

Подход 2. База данных + СУБД

Пользователь / программа

запрос на языке SQL

СУБД — система управления базами данных (например, SQLite)

↓

База данных: таблицы, связанные между собой

- ✓ Быстрый поиск запросом
- ✓ Один источник правды, нет копий
- ✓ Целостность: ключи и проверки
- ✓ Одновременно работают многие
- ✓ Данные хранятся надёжно

Место SQLite в семействе СУБД

MySQL	PostgreSQL	MS SQL Server	Oracle DB
серверная, для сайтов	серверная, мощная	серверная, от Microsoft	серверная, для корпораций

Серверные СУБД: нужен отдельный компьютер-сервер, настройка, сеть

SQLite — встраиваемая СУБД: вся база данных = ОДИН файл (university.sqlite)
Сервер не нужен — движок работает прямо внутри программы.
Самая распространённая СУБД в мире: есть в каждом телефоне и браузере

ВНИМАНИЕ. Новички часто путают два термина. База данных — это сами данные, аккуратно разложенные по таблицам. СУБД — это программа-библиотекарь, которая их хранит и выдаёт по запросу. Фраза «я создал базу в SQLite» означает: «я с помощью СУБД SQLite создал файл с данными».

Реляционная модель: данные — это таблицы

имена столбцов = заголовок таблицы

Таблица «Студенты» (students)

student_id	last_name	first_name	group_id
1	Иванов	Алексей	1
2	Петрова	Мария	1
3	Сидоров	Дмитрий	1
4	Кузнецова	Анна	2

СТОЛБЕЦ (поле, атрибут)
= одно свойство всех объектов

ЯЧЕЙКА = значение свойства
у конкретного объекта

СТРОКА (запись)
= один объект: один студент

Реляционная база данных = несколько таблиц, связанных между собой. «Реляционная» — от слова «отношение» (таблица и есть отношение). Связи строятся через общие столбцы-ключи — без дублирования данных.

Порядок строк в таблице не гарантирован — нужные строки находят по условиям, а не «на глаз»

ER-диаграмма учебной базы «Университет»

5 сущностей (таблиц) и связи между ними. PK — первичный ключ, FK — внешний ключ

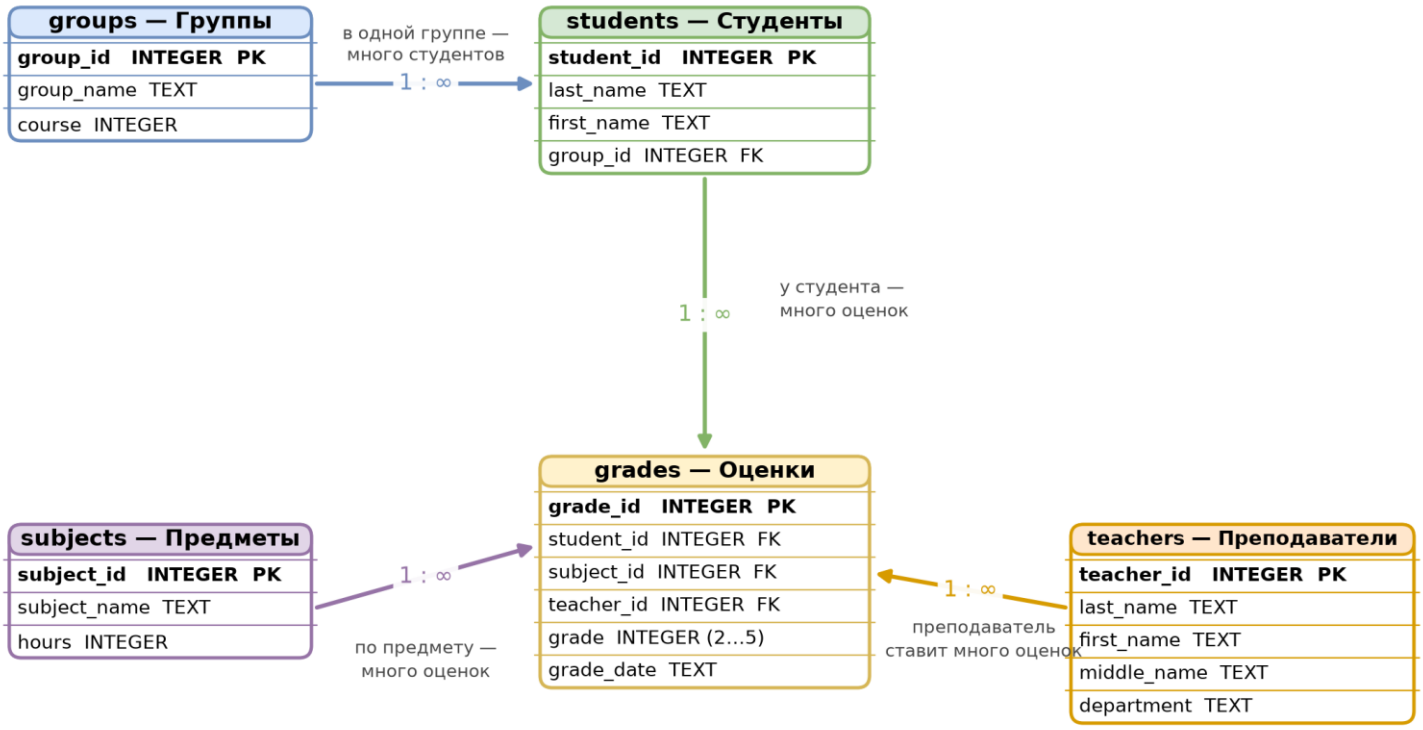


Таблица grades — связующая: у одной оценки ровно ОДИН студент, ОДИН предмет и ОДИН преподаватель. Такая связь называется «один ко многим» (1 : ∞) и реализуется внешними ключами (FK).

ПРОВЕРЬ СЕБЯ. 1) Чем база данных удобнее папки с файлами Excel? Назовите минимум два отличия.

2) Что такое СУБД и чем она отличается от базы данных?

3) Что в реляционной модели означает строка таблицы? А столбец?

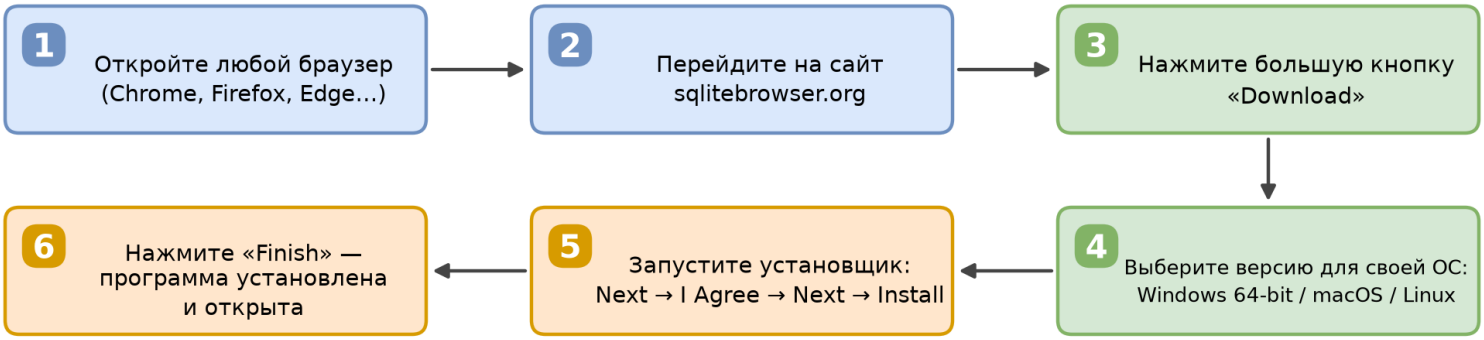
4) Как читается подпись «1 : ∞» на ER-диаграмме?

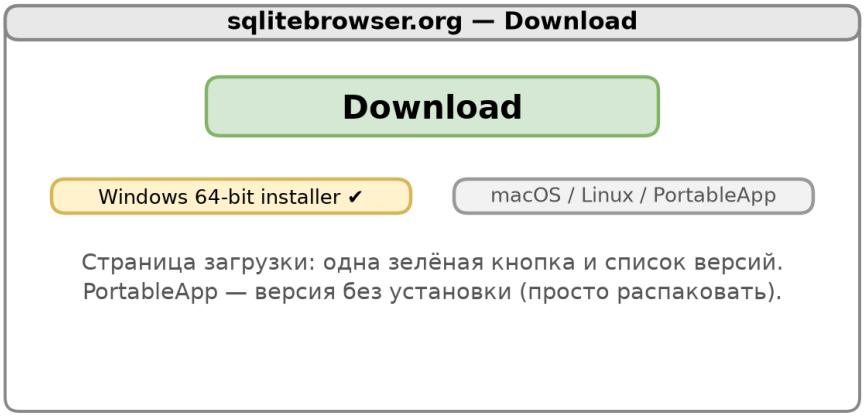
5) Почему название группы хранят в таблице groups, а не повторяют в каждой строке студентов?

Модуль 2. Установка DB Browser for SQLite

Теория: https://metanit.com/sql/sqlite/1.3.php

Установка DB Browser for SQLite: 6 простых шагов



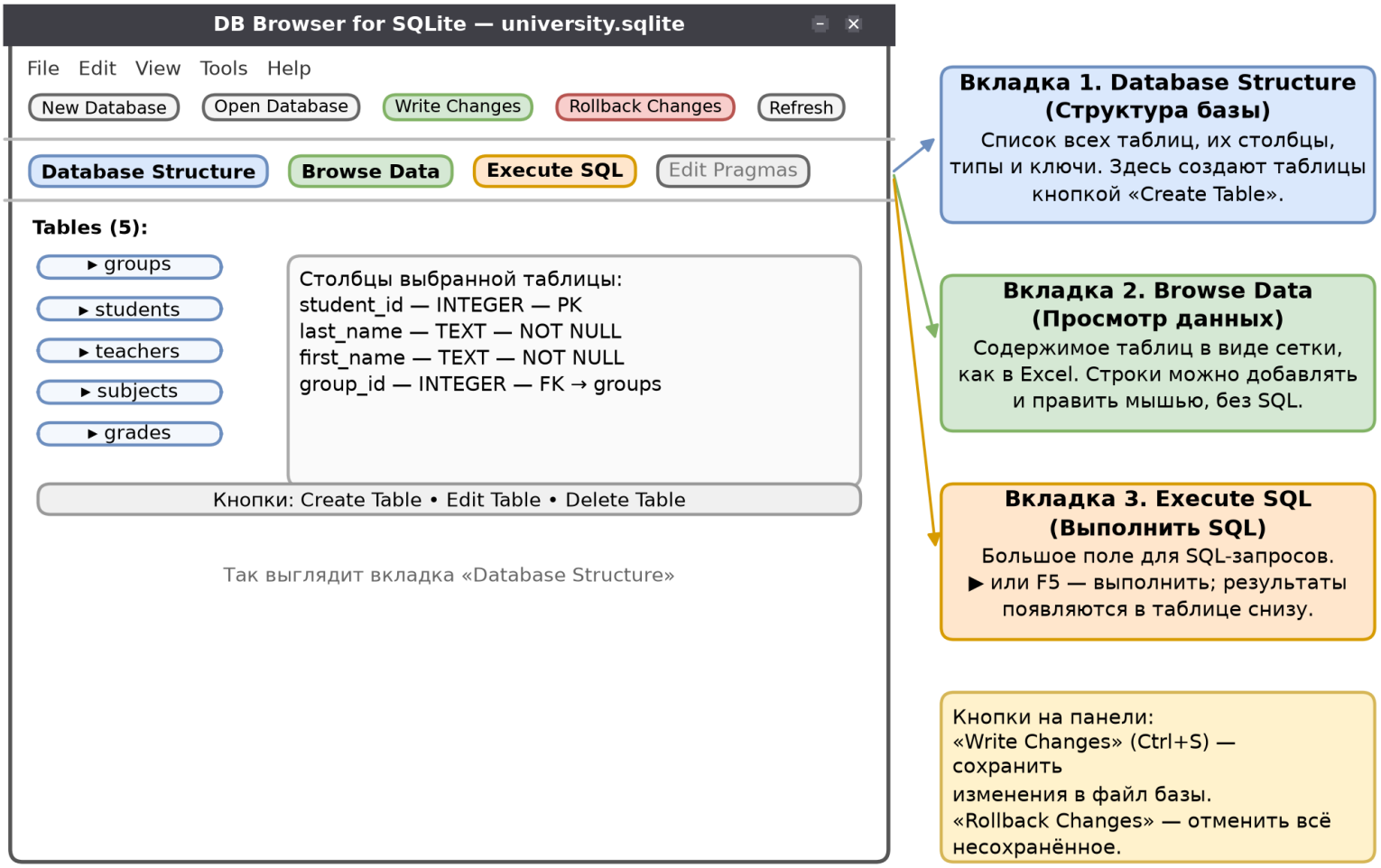


Если установка «ругается»
- Windows SmartScreen: «Подробнее» → «Выполнить»
- Путь установки оставьте по умолчанию
- Права администратора: нажмите «Да»

Полезно знать
- Программа бесплатная (open source)
- Интернет после установки не нужен
- Интерфейс переключается на русский: View → Preferences (или язык системы)

ПОДСКАЗКА. На учебном пк DB Browser for SQLite уже стоит. Если его нет, скачайте PortableApp-версию: это архив, который достаточно распаковать в свою папку (или на флешку) и запустить из него файл.

Интерфейс DB Browser: окно программы и 3 главные вкладки



Обратите внимание на две кнопки панели инструментов: «Write Changes» (сохранить изменения в файл, Ctrl+S) и «Rollback Changes» (отменить все несохранённые изменения). Пока изменения не сохранены, они живут только в памяти программы — это ваша страховка от ошибок.

2.3. Создаём файл базы данных

1. Создайте на диске папку для работы, например C:\db_practice (Windows) или ~/db_practice (Linux/macOS).
2. В DB Browser нажмите File → New Database (или кнопку «New Database»).
3. Программа спросит, куда сохранить файл. Выберите свою папку и укажите имя: university.sqlite. Нажмите Save.
4. Появится окно создания первой таблицы — пока просто нажмите Cancel (таблицы будем создавать в модуле 3 запросами SQL).
5. Убедитесь, что в заголовке окна появилось имя файла: «DB Browser for SQLite — university.sqlite». Пустая база готова!

ПРОВЕРЬ СЕБЯ. 1) С какого сайта скачивается DB Browser и платная ли это программа?

2) Какие три вкладки программы вы теперь знаете? Зачем каждая?

3) Что делают кнопки «Write Changes» и «Rollback Changes»?

4) Как называется файл вашей базы и где он лежит?

5) Чем PortableApp-версия отличается от обычной установки?

Модуль 3. Создание базы и таблиц: CREATE TABLE

Анатомия таблицы «Студенты»: как читать CREATE TABLE

```
① CREATE TABLE students (
②     student_id  INTEGER PRIMARY KEY AUTOINCREMENT,
③     last_name   TEXT    NOT NULL,
③     first_name  TEXT    NOT NULL,
④     group_id    INTEGER,
④     FOREIGN KEY (group_id) REFERENCES groups (group_id)
);
```

текст SQL-запроса, создающего таблицу

выполнить ▶

результат: таблица students

student_id	last_name	first_name	group_id
1	Иванов	Алексей	1
2	Петрова	Мария	1
3	Сидоров	Дмитрий	2

① Имя таблицы — students. По нему таблицу находят все запросы.

② student_id INTEGER PRIMARY KEY AUTOINCREMENT — первичный ключ: уникальный номер строки; SQLite присваивает его сам (1, 2, 3…).

③ last_name TEXT NOT NULL — столбец для текста; NOT NULL запрещает пустое значение.

④ group_id INTEGER + FOREIGN KEY … REFERENCES groups — внешний ключ: ссылка на таблицу groups, «в какой группе учится студент».

Тип	Для чего	Пример значения
INTEGER	Целые числа: ключи, количество, оценки	42, 5, −3
TEXT	Любой текст: фамилии, названия, даты в виде строки	'Иванов', '2026-09-05'
REAL	Дробные числа	4.75, 3.14
BLOB	Двоичные данные (картинки, файлы) — нам не понадобится	—

Кроме типа у столбца могут быть ограничения (constraints) — правила, которые СУБД соблюдает автоматически:

- PRIMARY KEY — первичный ключ: уникальное «имя» строки. Значения не повторяются и не бывают пустыми. AUTOINCREMENT добавляет к нему правило «SQLite сам выдаёт следующий номер».
- NOT NULL — значение в столбце обязательно, пустым (NULL) быть не может.
- UNIQUE — значения столбца не должны повторяться (например, название группы).
- DEFAULT — значение по умолчанию, если при вставке столбец не указан.
- CHECK — произвольная проверка значения (например, оценка от 2 до 5).
- FOREIGN KEY — внешний ключ: значение должно существовать в другой таблице. Так строятся связи.

3.2. Первичный и внешний ключ

Первичный ключ (PK) и внешний ключ (FK): как связываются таблицы

groups — «родитель»

group_id (PK)	group_name	course
1	ИС-21	2
2	ИС-22	1
3	ПМ-21	2
4	ЭК-23	1

students — «потомок»

student_id (PK)	last_name	first_name	group_id (FK)
1	Иванов	Алексей	1
2	Петрова	Мария	1
3	Сидоров	Дмитрий	1
4	Кузнецова	Анна	2
5	Смирнов	Игорь	2

Первичный ключ (PRIMARY KEY)
Уникальный «паспорт» строки. Значения не повторяются
и не бывают пустыми. Здесь — group_id: 1, 2, 3, 4.

Внешний ключ (FOREIGN KEY)
Столбец-ссылка: его значения берутся из PK другой таблицы.
Здесь — students.group_id ссылается на groups.group_id.

Правило целостности: в students.group_id нельзя записать номер группы, которой нет в groups.
SQLite может проверять это автоматически (PRAGMA foreign_keys = ON) — «сироты» в базе не появятся.

3.3. Практика: создаём пять таблиц

university.sql

C: > Users > Evgen > Downloads > university.sql

```sql

PRAGMA foreign_keys = ON;

-- Если таблицы уже есть — удаляем их, чтобы скрипт можно было
-- запускать повторно (сначала «дети», потом «родители»)
DROP TABLE IF EXISTS grades;
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS teachers;
DROP TABLE IF EXISTS subjects;
DROP TABLE IF EXISTS groups;

-- Таблица «Группы»
CREATE TABLE groups (
    group_id   INTEGER PRIMARY KEY AUTOINCREMENT, -- первичный ключ
    group_name TEXT    NOT NULL UNIQUE,           -- название группы, например ИС-21
    course     INTEGER NOT NULL DEFAULT 1         -- курс обучения
);

-- Таблица «Студенты»
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    last_name  TEXT    NOT NULL,                  -- фамилия
    first_name TEXT    NOT NULL,                  -- имя
    group_id   INTEGER,                           -- внешний ключ на groups
    FOREIGN KEY (group_id) REFERENCES groups (group_id)
);

-- Таблица «Преподаватели»
CREATE TABLE teachers (
    teacher_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    last_name    TEXT NOT NULL,
    first_name   TEXT NOT NULL,
    middle_name  TEXT,                            -- отчество (может отсутствовать)
    department   TEXT NOT NULL                    -- кафедра
);

```

university.sql

C: > Users > Evgen > Downloads > university.sql

```sql
-- Таблица «Предметы»
CREATE TABLE subjects (
    subject_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_name TEXT    NOT NULL UNIQUE,         -- название предмета
    hours        INTEGER NOT NULL DEFAULT 36      -- количество часов
);

-- Таблица «Оценки» — связывает студента, предмет и преподавателя
CREATE TABLE grades (
    grade_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    subject_id INTEGER NOT NULL,
    teacher_id INTEGER NOT NULL,
    grade      INTEGER NOT NULL CHECK (grade BETWEEN 2 AND 5),
    grade_date TEXT    NOT NULL,                  -- дата в формате ГГГГ-ММ-ДД
    FOREIGN KEY (student_id) REFERENCES students (student_id),
    FOREIGN KEY (subject_id) REFERENCES subjects (subject_id),
    FOREIGN KEY (teacher_id) REFERENCES teachers (teacher_id)
);
```

6. Создайте таблицы через Execute SQL или через СУБД (через СУБД приоритетнее)

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    last_name  TEXT    NOT NULL,
    first_name TEXT    NOT NULL,
    group_id   INTEGER,
    FOREIGN KEY (group_id) REFERENCES groups (group_id)
);
```

ПРОВЕРЬ СЕБЯ. 1) Какой тип данных вы выберете для: возраста студента, его фамилии, средней оценки, даты сдачи экзамена?

2) Зачем нужен PRIMARY KEY? Что добавляет AUTOINCREMENT?

3) Чем NOT NULL отличается от UNIQUE?

4) Что проверяет FOREIGN KEY и зачем он нужен в таблице grades (их там целых три)?

5) Сколько таблиц теперь видно на вкладке Database Structure?

Модуль 4. Наполнение данными: INSERT

4.1. Как устроен INSERT

Пустая структура бесполезна — базе нужны данные. Строки добавляет запрос INSERT. Его общая форма:

```
INSERT INTO имя_таблицы (столбец1, столбец2, ...) VALUES
    (значение1, значение2, ...);
```

4.2. Практика: загружаем данные университета

```
-- Группы
INSERT INTO groups (group_name, course) VALUES
    ('ИС-21', 2),
    ('ИС-22', 1),
    ('ПМ-21', 2),
    ('ЭК-23', 1);

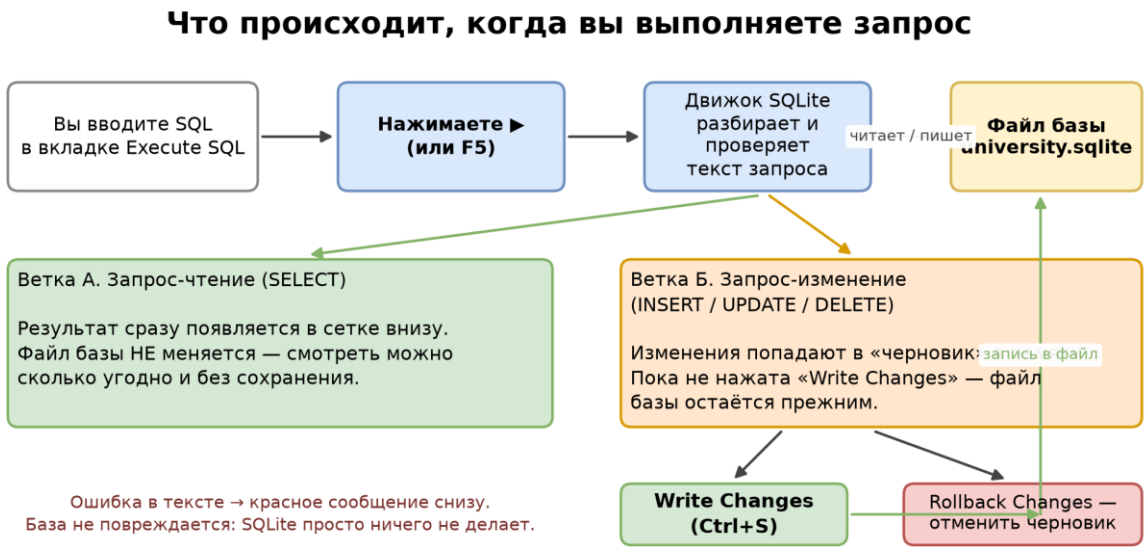
-- Студенты
INSERT INTO students (last_name, first_name, group_id) VALUES
    ('Иванов',   'Алексей',  1),
    ('Петрова',  'Мария',    1),
    ('Сидоров',  'Дмитрий',  1),
    ('Кузнецова','Анна',     2),
    ('Смирнов',  'Игорь',    2),
    ('Волкова',  'Елена',    2),
    ('Морозов',  'Павел',    3),
    ('Зайцева',  'Ольга',    3),
    ('Лебедев',  'Никита',   3),
    ('Козлова',  'Дарья',    4),
    ('Новиков',  'Артём',    4),
    ('Соколова', 'Виктория', 1);

-- Преподаватели
INSERT INTO teachers (last_name, first_name, middle_name, department) VALUES
    ('Андреева', 'Галина',  'Петровна',   'Информатика'),
    ('Борисов',  'Сергей',  'Иванович',   'Математика'),
    ('Ветрова',  'Наталья', 'Андреевна',  'Экономика'),
    ('Громов',   'Илья',    NULL,         'Информатика');

-- Предметы
INSERT INTO subjects (subject_name, hours) VALUES
    ('Базы данных',           72),
    ('Математический анализ', 108),
    ('Программирование',      90),
    ('Экономика',             54),
    ('Английский язык',       72);

-- Оценки
INSERT INTO grades (student_id, subject_id, teacher_id, grade, grade_date) VALUES
    (1, 1, 1, 5, '2026-09-05'),
    (1, 3, 4, 4, '2026-09-12'),
    (2, 1, 1, 4, '2026-09-05'),
    (2, 2, 2, 5, '2026-09-08'),
    (3, 1, 1, 3, '2026-09-05'),
    (3, 3, 4, 2, '2026-09-12'),
    (4, 1, 1, 5, '2026-09-06'),
    (4, 2, 2, 5, '2026-09-09'),
    (5, 2, 2, 3, '2026-09-09'),
    (5, 3, 4, 4, '2026-09-13'),
    (6, 1, 1, 4, '2026-09-06'),
    (7, 2, 2, 4, '2026-09-10'),
    (7, 3, 4, 5, '2026-09-13'),
    (8, 2, 2, 5, '2026-09-10'),
    (9, 3, 4, 3, '2026-09-14'),
    (10, 4, 3, 4, '2026-09-11'),
    (11, 4, 3, 3, '2026-09-11'),
    (12, 1, 1, 5, '2026-09-06'),
    (12, 2, 2, 4, '2026-09-10');
```

```
INSERT INTO students (last_name, first_name, group_id) VALUES
    ('Иванов',   'Алексей',  1),
    ('Петрова',  'Мария',    1),
    ('Сидоров',  'Дмитрий',  1),
    ('Кузнецова','Анна',     2);
```



ПРОВЕРЬ СЕБЯ. 1) Сколько строк теперь в каждой из пяти таблиц? (Проверьте себя по Browse Data.)

2) Почему значения 'Иванов' и '2026-09-05' в кавычках, а число 1 — без?

3) Что произойдёт, если после INSERT закрыть программу, не нажав «Write Changes»?

4) Зачем в INSERT перечислять столбцы явно, если можно просто перечислить значения?

5) Как добавить несколько строк одним запросом?

Модуль 5. Выборка данных: SELECT, WHERE, ORDER BY, LIMIT

5.1. Первый SELECT

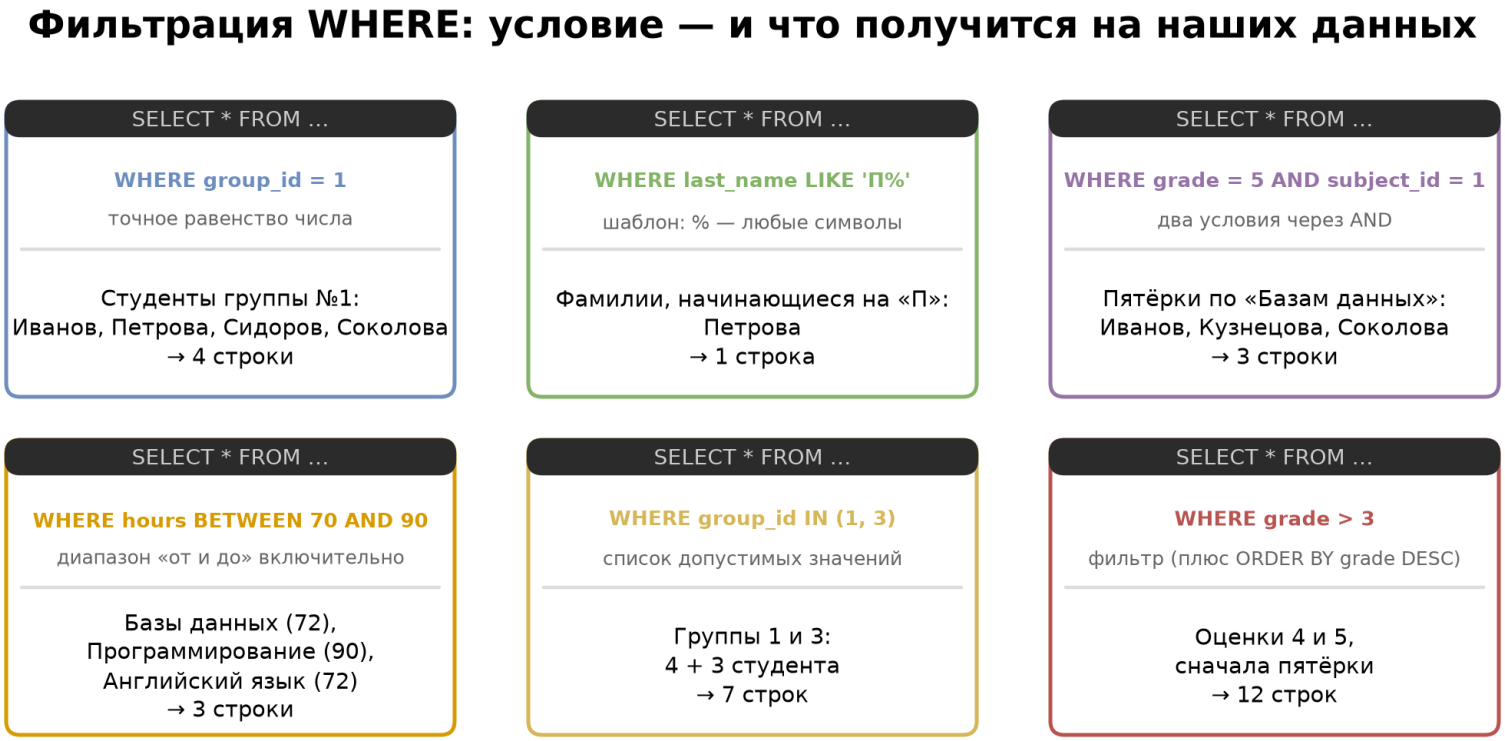
SELECT — самый частый запрос в SQL: он ничего не меняет, только показывает данные. Минимальный SELECT состоит из двух частей: ЧТО взять и ОТКУДА.

```
SELECT * FROM students;                          -- все столбцы, все строки
SELECT last_name, first_name FROM students;      -- только два столбца
SELECT last_name AS Фамилия, first_name AS Имя
FROM students;                                   -- AS задаёт псевдоним столбца
```

Звёздочка * означает «все столбцы». В реальной работе её стараются не использовать: перечисляйте только нужные столбцы — так результат понятнее, а запрос быстрее.

5.2. Фильтрация: WHERE

WHERE оставляет только строки, удовлетворяющие условию. Условий много — основные собраны на рисунке 10 с результатами на данных нашей базы.



Как читать результат: WHERE оставляет ТОЛЬКО строки, для которых условие истинно. Сравнение текста — в одинарных кавычках ('П%'), чисел — без кавычек. LIKE не чувствителен к регистру для латиницы, а для кириллицы регистр учитывайте: 'п%' и 'П%' — разные шаблоны.

Проверьте сами в DB Browser: вкладка Execute SQL → вставьте карточку целиком: SELECT * FROM students WHERE group_id = 1; → нажмите ▶ → сравните с сеткой Browse Data. Каждую карточку можно превратить в запрос, добавив в начало SELECT * FROM <таблица>.

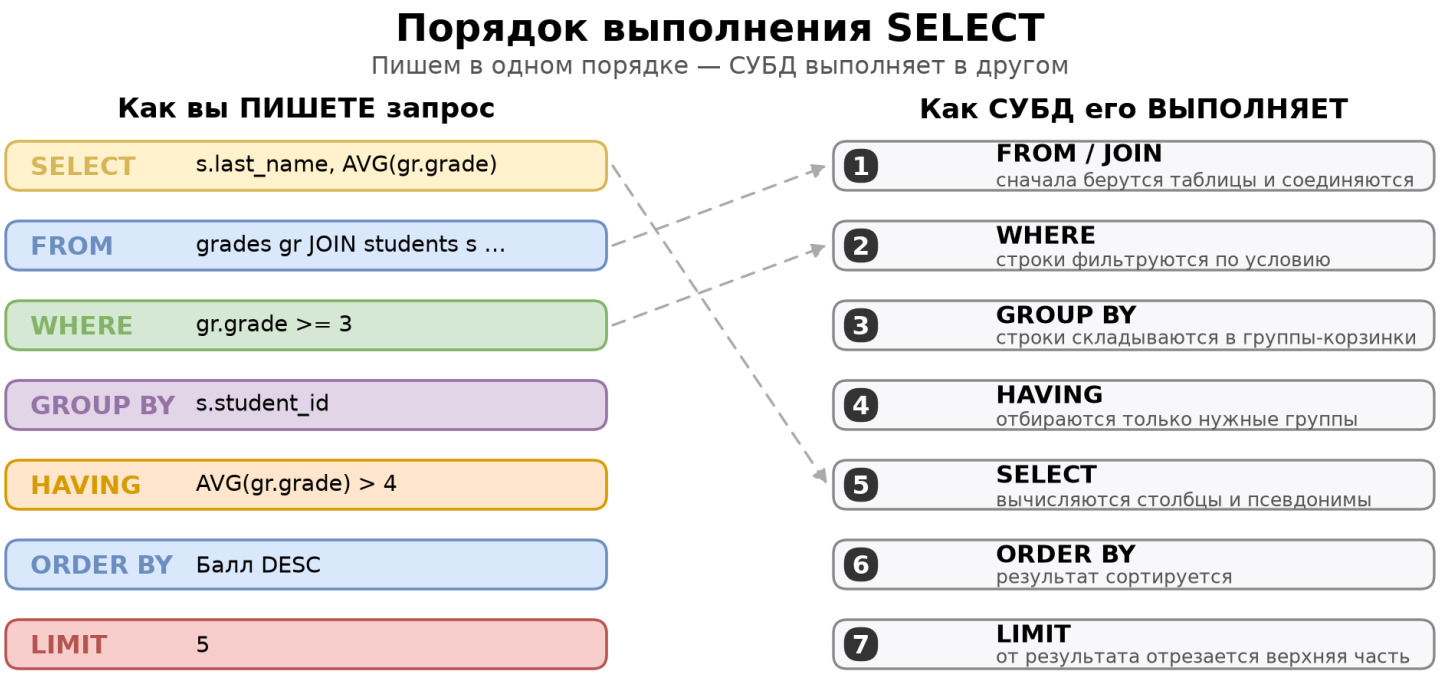
Оператор / конструкция	Что делает	Пример
= > < >= <= <>	Сравнение значений	grade = 5, hours > 60
AND / OR / NOT	Объединение условий	grade = 5 AND subject_id = 1
BETWEEN a AND b	Диапазон включительно	grade BETWEEN 4 AND 5
IN (…)	Входит в список	group_id IN (1, 3)
LIKE 'шаблон'	Похоже на шаблон (% — любые символы, _ — один символ)	last_name LIKE 'П%'
IS NULL / IS NOT NULL	Пусто / не пусто	middle_name IS NULL

5.3. Сортировка и ограничение: ORDER BY и LIMIT

```
-- отсортировать по группе, а внутри группы — по фамилии
SELECT last_name, first_name, group_id FROM students
ORDER BY group_id, last_name;

-- самые свежие оценки первыми, и только пять штук
SELECT * FROM grades ORDER BY grade_date DESC LIMIT 5;
```

5.4. В каком порядке СУБД выполняет SELECT



Почему это важно знать: на шаге WHERE псевдонимов из SELECT ещё не существует — поэтому в WHERE пишут AVG(grade), а не «Балл». А HAVING существует именно потому, что фильтр по группам нужен ПОСЛЕ группировки, когда WHERE уже отработал.

Запомнить просто: «Фри WHERE Группу Имеет — Select Order Limit»

5.5. Практика

```sql
-- 5.1. Все столбцы и все строки таблицы
SELECT * FROM students;

-- 5.2. Только нужные столбцы
SELECT last_name, first_name FROM students;

-- 5.3. Столбцы с русскими псевдонимами (AS)
SELECT last_name AS Фамилия, first_name AS Имя, group_id AS Группа
FROM students;

-- 5.4. Фильтр WHERE: точное равенство
SELECT * FROM students WHERE group_id = 1;

-- 5.5. Сравнение чисел
SELECT * FROM subjects WHERE hours > 60;

-- 5.6. Текстовый поиск с шаблоном LIKE (% — любая последовательность)
SELECT * FROM students WHERE last_name LIKE 'И%';

-- 5.7. Несколько условий: AND / OR / NOT
SELECT * FROM grades WHERE grade = 5 AND subject_id = 1;
SELECT * FROM students WHERE group_id = 1 OR group_id = 2;
SELECT * FROM grades WHERE NOT grade = 5;

-- 5.8. Диапазон значений: BETWEEN и IN
SELECT * FROM grades WHERE grade BETWEEN 4 AND 5;
SELECT * FROM students WHERE group_id IN (1, 3);

-- 5.9. Сортировка ORDER BY (DESC — по убыванию)
SELECT last_name, first_name, group_id
FROM students
ORDER BY group_id, last_name;

SELECT * FROM grades ORDER BY grade DESC, grade_date;

-- 5.10. Ограничение числа строк LIMIT
SELECT * FROM grades ORDER BY grade_date LIMIT 5;

```

Задания модуля 5 (сдаются преподавателю)

Задание 5А. Выведите все предметы, у которых больше 60 часов, отсортировав по убыванию часов.

Задание 5Б. Выведите фамилии и имена студентов группы «ИС-21» по алфавиту. (Подсказка: сначала узнайте group_id этой группы запросом SELECT * FROM groups; — или сделайте «в лоб» по group_id.)

ПРОВЕРЬ СЕБЯ. 1) Что означает SELECT *?

2) Как вывести только первые 10 строк таблицы grades?

3) Чем BETWEEN 4 AND 5 отличается от grade > 4 AND grade < 5?

4) Что делает шаблон LIKE '_ван%'?

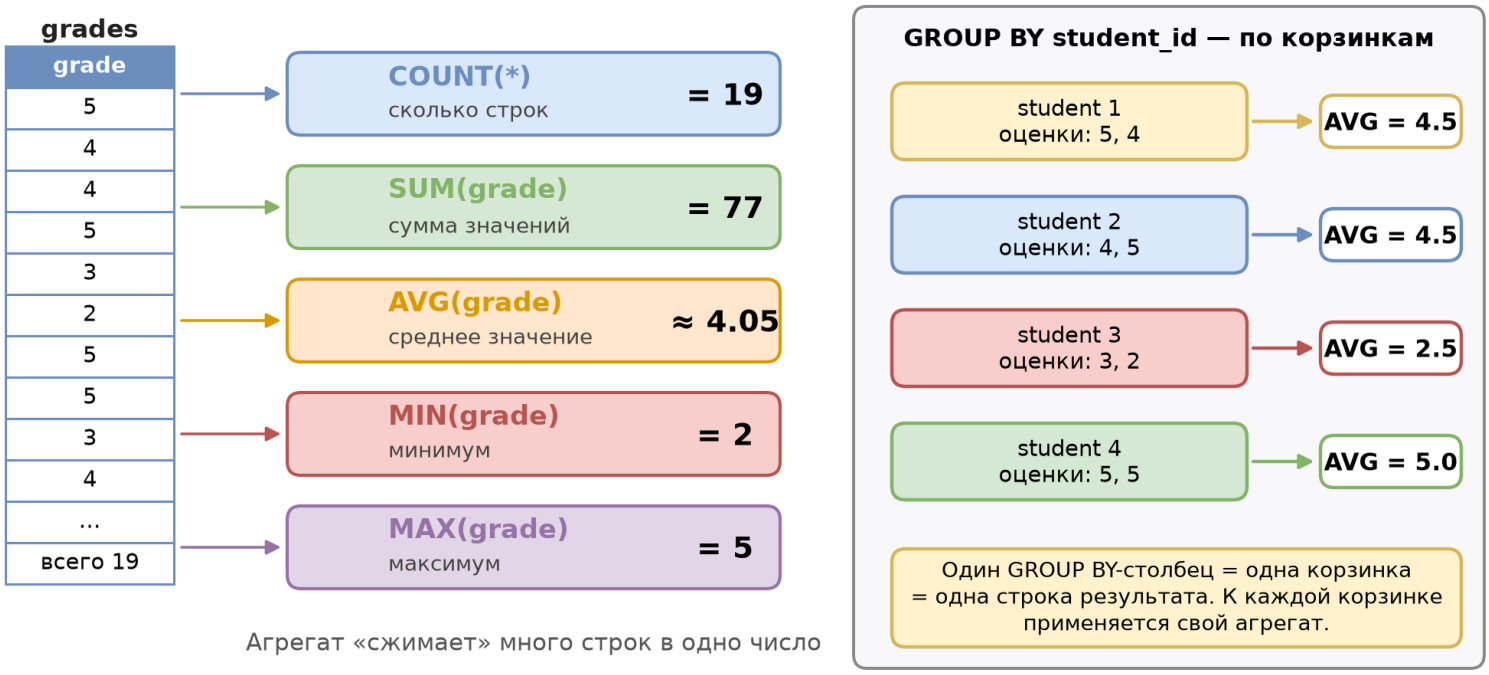
5) В каком порядке SQLite выполняет части запроса SELECT?

Модуль 6. Агрегаты и группировка: COUNT, AVG, SUM, GROUP BY

6.1. Пять агрегатных функций

Агрегатная функция «сжимает» много значений в одно число. Пять главных агрегатов SQL:

Функция	Что считает	Пример на нашей базе (таблица grades, 19 оценок)
COUNT(*)	Количество строк	COUNT(*) = 19 — всего оценок
SUM(столбец)	Сумму значений	SUM(grade) = 77
AVG(столбец)	Среднее значение	AVG(grade) ≈ 4.05
MIN(столбец)	Минимальное значение	MIN(grade) = 2
MAX(столбец)	Максимальное значение	MAX(grade) = 5



Полный запрос: SELECT student_id, AVG(grade) AS Средний_балл, COUNT(*) AS Оценок
FROM grades GROUP BY student_id ORDER BY Средний_балл DESC;
Правило: в SELECT рядом с агрегатами можно ставить только те столбцы, по которым идёт GROUP BY.

Рисунок 12. Агрегатные функции и GROUP BY

```
SELECT COUNT(*) AS Всего_студентов FROM students;

SELECT AVG(grade) AS Средний_балл,
       MIN(grade) AS Минимум,
       MAX(grade) AS Максимум
FROM grades;
```

6.2. GROUP BY: агрегаты по группам

«Средняя оценка по всей базе» — не очень полезная цифра. Интереснее средний балл каждого студента. Для этого строки сначала раскладывают по «корзинкам» (GROUP BY student_id — у каждого студента своя корзинка), а потом агрегат считают в каждой корзинке:

```
SELECT student_id,
       AVG(grade)   AS Средний_балл,
       COUNT(*)     AS Число_оценок
FROM grades
GROUP BY student_id
ORDER BY Средний_балл DESC;
```

6.3. HAVING: фильтр по группам

WHERE фильтрует строки до группировки, а HAVING — готовые группы после. Сравните: «оценки выше тройки» — это WHERE grade > 3; «студенты, у которых больше одной оценки» — это HAVING COUNT(*) > 1.

```
SELECT student_id, COUNT(*) AS Число_оценок
FROM grades
GROUP BY student_id
HAVING COUNT(*) > 1;
```

6.4. DISTINCT и ROUND

```
SELECT DISTINCT grade FROM grades;          -- какие вообще оценки встречаются
SELECT DISTINCT group_id FROM students;     -- в каких группах есть студенты
SELECT ROUND(AVG(grade), 2) FROM grades;    -- среднее с округлением до 2 знаков
```

6.5. Практика

```sql
-- 6.1. COUNT — сколько всего строк
SELECT COUNT(*) AS Всего_студентов FROM students;

-- 6.2. AVG, MIN, MAX, SUM
--      (агрегат применяется к столбцу той таблицы, что указана в FROM)
SELECT AVG(grade) AS Средний_балл,
       MIN(grade) AS Минимум,
       MAX(grade) AS Максимум
FROM grades;

SELECT SUM(hours) AS Всего_часов FROM subjects;

-- 6.3. GROUP BY — средний балл по каждому студенту
SELECT student_id, AVG(grade) AS Средний_балл, COUNT(*) AS Число_оценок
FROM grades
GROUP BY student_id;

-- 6.4. GROUP BY + ORDER BY: средний балл по предметам, от высшего к низшему
SELECT subject_id, AVG(grade) AS Средний_балл
FROM grades
GROUP BY subject_id
ORDER BY Средний_балл DESC;

-- 6.5. HAVING — фильтр по результату группировки
--      (только студенты, у которых больше одной оценки)
SELECT student_id, COUNT(*) AS Число_оценок
FROM grades
GROUP BY student_id
HAVING COUNT(*) > 1;

-- 6.6. DISTINCT — уникальные значения
SELECT DISTINCT grade FROM grades;
SELECT DISTINCT group_id FROM students;
```

Задания модуля 6 (сдаются преподавателю)

Задание 6А. Сколько студентов в каждой группе? Выведите group_id и количество.

Задание 6Б. Вычислите среднюю оценку по предмету «Базы данных» (subject_id = 1).

ПРОВЕРЬ СЕБЯ. 1) Какой агрегат ответит на вопрос «сколько всего студентов»?

2) Что вернёт SELECT AVG(grade) FROM grades; и почему результат дробный?

3) Зачем нужен GROUP BY и что он делает со строками?

4) Чем HAVING отличается от WHERE?

5) Что покажет SELECT DISTINCT group_id FROM students;?

Модуль 7. Соединение таблиц: JOIN

```
SELECT s.last_name, s.first_name, g.group_name
FROM students s
INNER JOIN groups g ON s.group_id = g.group_id
ORDER BY g.group_name, s.last_name;
```

7.2. INNER против LEFT



Запросы 7.1–7.6 из university.sql выполните в DB Browser и сравните число строк в результатах.

```
-- предметы, по которым ещё никто не получил оценку
SELECT sub.subject_name
FROM subjects sub
LEFT JOIN grades gr ON gr.subject_id = sub.subject_id
WHERE gr.grade_id IS NULL;
```

7.3. Несколько JOIN подряд

Таблицы можно соединять цепочкой — так строится полный отчёт об оценках из четырёх таблиц: студент → группа, оценка → предмет, оценка → преподаватель.

```
SELECT s.last_name  AS Студент,
       g.group_name AS Группа,
       sub.subject_name AS Предмет,
```

```
       t.last_name  AS Преподаватель,
       gr.grade     AS Оценка,
       gr.grade_date AS Дата
FROM grades gr
JOIN students s   ON gr.student_id = s.student_id
JOIN groups g     ON s.group_id    = g.group_id
JOIN subjects sub ON gr.subject_id = sub.subject_id
JOIN teachers t   ON gr.teacher_id = t.teacher_id
ORDER BY gr.grade_date;
```

Слово JOIN без уточнения означает то же самое, что INNER JOIN. Цепочка читается просто: к таблице grades присоединили students, к получившемуся — groups, и так далее. JOIN отлично сочетается с агрегатами и GROUP BY — так получают «средний балл каждого студента с фамилией»:

```
SELECT s.last_name, s.first_name, AVG(gr.grade) AS Средний_балл
FROM grades gr
JOIN students s ON gr.student_id = s.student_id
GROUP BY s.student_id
ORDER BY Средний_балл DESC;
```

7.4. Практика

```sql
-- 7.1. INNER JOIN — только совпавшие строки:
--      фамилии студентов вместе с названием их группы
SELECT s.last_name, s.first_name, g.group_name
FROM students s
INNER JOIN groups g ON s.group_id = g.group_id
ORDER BY g.group_name, s.last_name;

-- 7.2. LEFT JOIN — все группы, даже те, где пока нет студентов
SELECT g.group_name, s.last_name
FROM groups g
LEFT JOIN students s ON s.group_id = g.group_id
ORDER BY g.group_name;

-- 7.3. JOIN трёх таблиц: студент — оценка — предмет
SELECT s.last_name, sub.subject_name, gr.grade, gr.grade_date
FROM grades gr
JOIN students s ON gr.student_id = s.student_id
JOIN subjects sub ON gr.subject_id = sub.subject_id
ORDER BY s.last_name, gr.grade_date;

-- 7.4. JOIN четырёх таблиц: полный отчёт об оценках
SELECT s.last_name  AS Студент,
       g.group_name AS Группа,
       sub.subject_name AS Предмет,
       t.last_name  AS Преподаватель,
       gr.grade     AS Оценка,
       gr.grade_date AS Дата
FROM grades gr
JOIN students s   ON gr.student_id = s.student_id
JOIN groups g     ON s.group_id    = g.group_id
JOIN subjects sub ON gr.subject_id = sub.subject_id
JOIN teachers t   ON gr.teacher_id = t.teacher_id
ORDER BY gr.grade_date;

-- 7.5. LEFT JOIN + IS NULL: предметы, по которым ещё нет оценок
SELECT sub.subject_name
FROM subjects sub
LEFT JOIN grades gr ON gr.subject_id = sub.subject_id
WHERE gr.grade_id IS NULL;

-- 7.6. Средний балл с именами (агрегат + JOIN + GROUP BY)
SELECT s.last_name, s.first_name, AVG(gr.grade) AS Средний_балл
FROM grades gr
JOIN students s ON gr.student_id = s.student_id
GROUP BY s.student_id
ORDER BY Средний_балл DESC;
```

Задание модуля 7 (сдаётся преподавателю)

Задание 7А. Составьте список «преподаватель — предмет — сколько оценок он поставил по этому предмету», отсортированный по фамилии преподавателя.

ПРОВЕРЬ СЕБЯ. 1) Зачем в запросах псевдонимы таблиц (students s)?

2) Что произойдёт, если в JOIN забыть условие ON?

3) Чем LEFT JOIN отличается от INNER JOIN? Приведите жизненный пример вопроса, который решает LEFT JOIN.

4) Как найти предметы, по которым нет ни одной оценки?

5) Можно ли использовать AVG вместе с JOIN и GROUP BY? Как?

Модуль 8. Изменение и удаление: UPDATE, DELETE. Итог (1 час)

8.1. UPDATE: изменение строк

```
-- перевести студента №3 во вторую группу
UPDATE students SET group_id = 2 WHERE student_id = 3;

-- поднять курс сразу всем группам
UPDATE groups SET course = course + 1;
```

UPDATE меняет существующие строки: SET задаёт новые значения столбцов, WHERE — какие строки менять. Во втором примере WHERE нет намеренно: поменять нужно все строки. Но это редкий случай!

8.2. DELETE: удаление строк

```
-- удалить одну конкретную оценку
DELETE FROM grades WHERE grade_id = 3;

-- удалить все двойки
DELETE FROM grades WHERE grade = 2;
```

DELETE удаляет строки целиком (нельзя «удалить значение одного столбца» — для этого есть UPDATE … SET столбец = NULL). Удаляя строку-«родителя», помните о детях: пока включена проверка внешних ключей, SQLite не даст удалить группу, в которой есть студенты, — и будет прав.

8.3. Сохранение и откат

Изменения, сделанные INSERT/UPDATE/DELETE, не попадают в файл автоматически. Кнопка «Write Changes» (Ctrl+S) записывает их в university.sqlite, а «Rollback Changes» — отменяет всё несохранённое. Это полноценная страховка: ошиблись — откатились, и база снова в порядке.

8.4. Практика

```sql
-- 8.1. UPDATE — изменить одну строку (ОБЯЗАТЕЛЬНО с WHERE!)
UPDATE students SET group_id = 2 WHERE student_id = 3;

-- 8.2. UPDATE — массовое изменение: поднять курс всем группам
UPDATE groups SET course = course + 1;

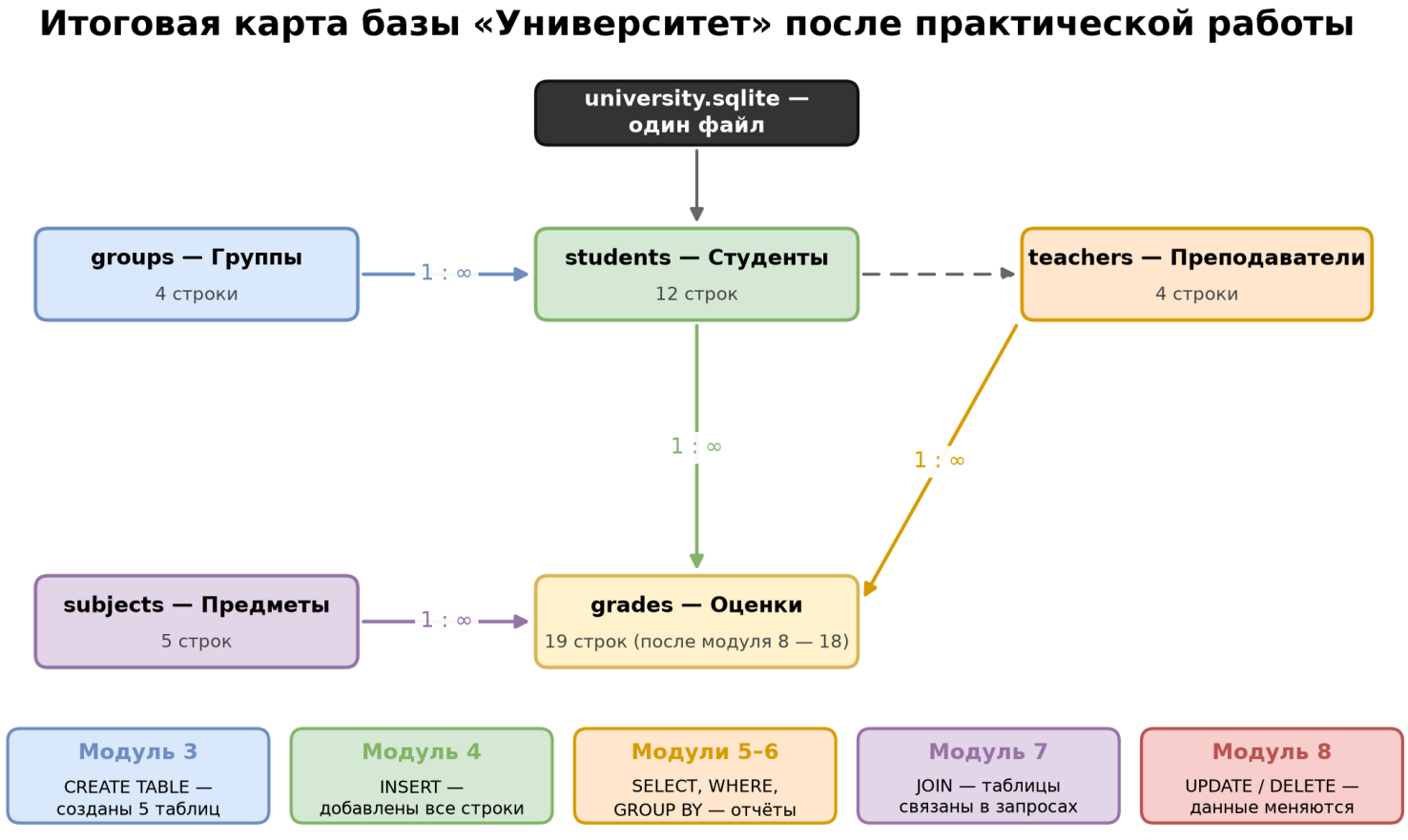
-- 8.3. DELETE — удалить одну строку (тоже с WHERE!)
DELETE FROM grades WHERE grade_id = 3;

-- 8.4. INSERT после изменений: вернуть удалённую оценку
INSERT INTO grades (student_id, subject_id, teacher_id, grade, grade_date)
VALUES (3, 1, 1, 4, '2026-09-05');

-- 8.5. Откат примеров 8.1–8.2, чтобы база осталась в исходном виде
UPDATE students SET group_id = 1 WHERE student_id = 3;
UPDATE groups SET course = course - 1;

-- ВАЖНО: после UPDATE/DELETE нажмите «Write Changes» (Ctrl+S)
-- или «Rollback Changes», чтобы отменить несохранённое.
```

8.5. Итоговая картина



Всё это живёт в одном файле university.sqlite — его можно перенести на флешке и открыть на любом компьютере

8.6. Если что-то пошло не так

Ошибки будут — это часть обучения. Главное — уметь их читать: SQLite всегда сообщает причину дословно. Пользуйтесь блок-схемой диагностики:



Рисунок 15. Диагностика ошибок: что делать, если запрос не работает

ПРОВЕРЬ СЕБЯ. 1) Из каких двух частей состоит UPDATE и за что каждая отвечает?

2) Почему UPDATE/DELETE без WHERE называют «опасными»?

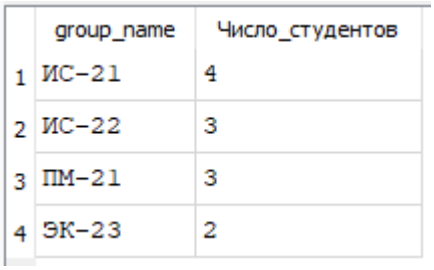
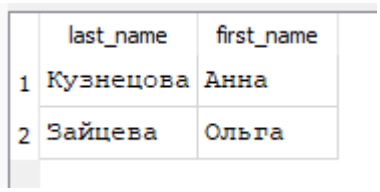
3) Как заранее узнать, какие строки затронет DELETE?

4) Что делает «Rollback Changes»?

5) Ошибка «FOREIGN KEY constraint failed» при DELETE — о чём она говорит?

Итоговое самостоятельное задание

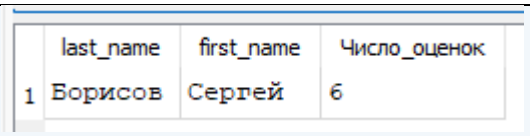
Перед началом убедитесь, что база в исходном состоянии: при необходимости заново выполните блоки модулей 3–4.

№	Задание	Какие темы проверяет
1	Сколько студентов в каждой группе? Вывести название группы и число студентов, отсортировать по убыванию числа. 	JOIN, GROUP BY, COUNT, ORDER BY
2	Средний балл каждого студента: фамилия, имя, группа, балл (округлить до 2 знаков). Отсортировать по убыванию балла. 	JOIN трёх таблиц, AVG, ROUND, GROUP BY
3	Список отличников — студентов, у которых средний балл ровно 5. 	GROUP BY, HAVING
4	Какой преподаватель поставил больше всего оценок? Вывести фамилию, имя и число оценок.	JOIN, COUNT, GROUP BY, ORDER BY, LIMIT

	group_name	Число_студентов
1	ИС-21	4
2	ИС-22	3
3	ПМ-21	3
4	ЭК-23	2

	last_name	first_name	group_name	Средний_балл
1	Зайцева	Ольга	ПМ-21	5.0
2	Кузнецова	Анна	ИС-22	5.0
3	Соколова	Виктория	ИС-21	4.5
4	Морозов	Павел	ПМ-21	4.5
5	Петрова	Мария	ИС-21	4.5
6	Иванов	Алексей	ИС-21	4.5
7	Козлова	Дарья	ЭК-23	4.0
8	Волкова	Елена	ИС-22	4.0
9	Смирнов	Игорь	ИС-22	3.5
10	Новиков	Артём	ЭК-23	3.0
11	Лебедев	Никита	ПМ-21	3.0
12	Сидоров	Дмитрий	ИС-21	2.5

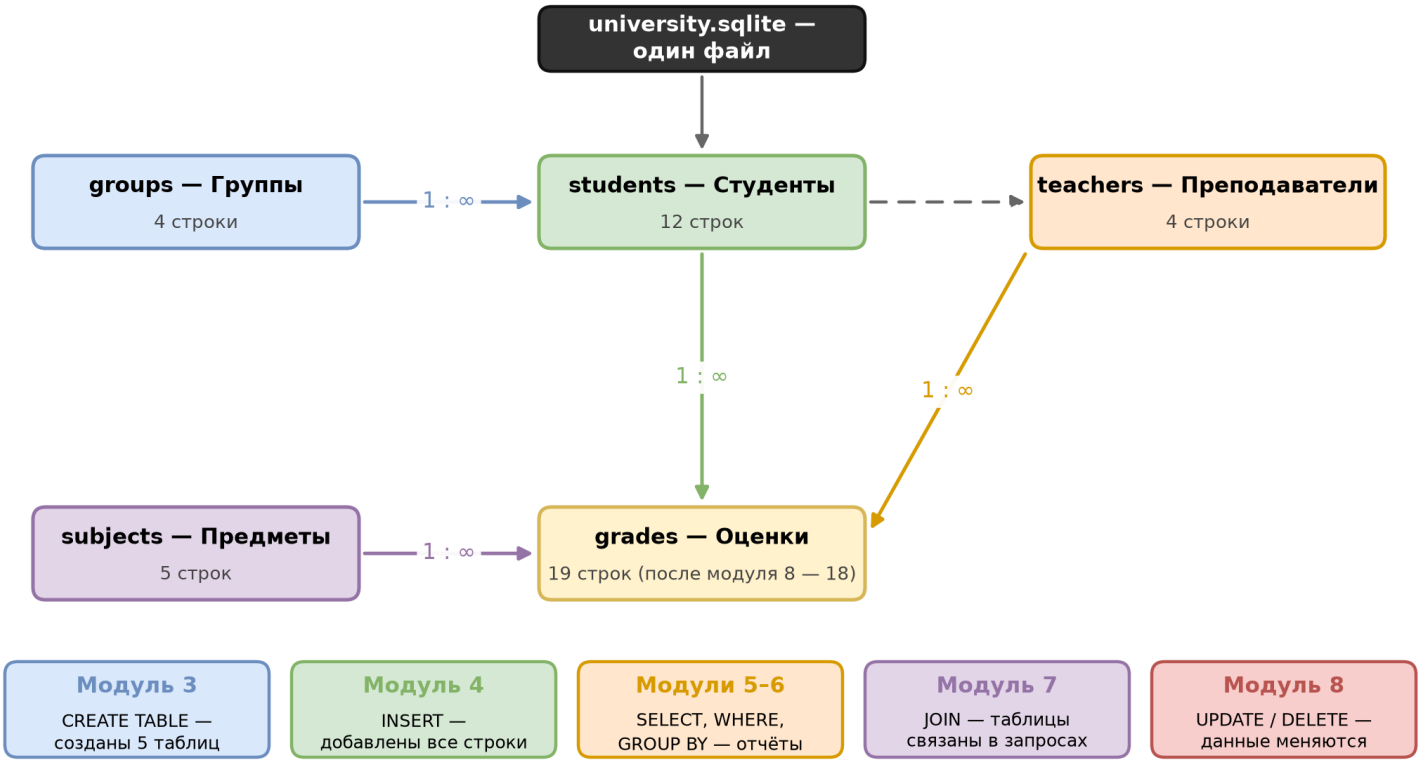
	last_name	first_name
1	Кузнецова	Анна
2	Зайцева	Ольга

	last_name	first_name	Число_оценок
1	Борисов	Сергей	6

Схема всей практической работы

Итоговая карта базы «Университет» после практической работы



Всё это живёт в одном файле university.sqlite — его можно перенести на флешке и открыть на любом компьютере

Практическая работа Python + Базы данных

Создайте папку pythonDB. Скопируйте в нее базу данных из прошлой работ. И выполните следующий код.

Теория: https://metanit.com/python/database/1.1.php

main.py > ...

```python
import sqlite3

con = sqlite3.connect("test.db")

cursor = con.cursor()

rows = cursor.execute("SELECT * FROM groups").fetchall()

print(rows)
for row in rows:
    print(row)
```

Результат:

```
PS C:\Users\Evgen\Desktop\pythonDB> python .\main.py
[(1, 'ИС-21', 2), (2, 'ИС-22', 1), (3, 'ПМ-21', 2), (4, 'ЭК-23', 1)]
(1, 'ИС-21', 2)
(2, 'ИС-22', 1)
(3, 'ПМ-21', 2)
(4, 'ЭК-23', 1)
```

Подключаем более современные инструменты работы с Базами данных в python под названием SQLAlchemy

Теория: https://metanit.com/python/database/3.1.php

- uv init
- uv add sqlalchemy

```
PS C:\Users\Evgen\Desktop\pythonDB> uv init
Initialized project `pythondb`
PS C:\Users\Evgen\Desktop\pythonDB> uv add SQLAlchemy
Using CPython 3.12.11
Creating virtual environment at: .venv
Resolved 3 packages in 8.42s
      Built pythondb @ file:///C:/Users/Evgen/Desktop/pythonDB
Prepared 2 packages in 766ms
Installed 3 packages in 249ms
 + pythondb==0.1.0 (from file:///C:/Users/Evgen/Desktop/pythonDB)
 + sqlalchemy==2.1.6
 + typing-extensions==4.16.0
PS C:\Users\Evgen\Desktop\pythonDB>
```

Результат:

- Теперь сделаем тоже самое но с новой либой:

main.py > ...

```python
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import Session, DeclarativeBase

engine = create_engine("sqlite:///test.db")

class Base(DeclarativeBase): pass
class Groups(Base):
    __tablename__ = "groups"

    group_id = Column(Integer, primary_key=True, index=True)
    group_name = Column(String)
    course = Column(Integer)


with Session(autoflush=False, bind=engine) as db:
    groups = db.query(Groups).all()
    for group in groups:
        print(f"ID:{group.group_id} Название группы: {group.group_name} Курс: {group.course}")
```

```
PS C:\Users\Evgen\Desktop\pythonDB> uv run main.py
ID:1 Название группы: ИС-21 Курс: 2
ID:2 Название группы: ИС-22 Курс: 1
ID:3 Название группы: ПМ-21 Курс: 2
ID:4 Название группы: ЭК-23 Курс: 1
PS C:\Users\Evgen\Desktop\pythonDB>
```

Результат:

- Кажется что это сделало код более сложным, но зато мы полностью избавились от SQL в нашем коде и значительно улучшили безопасность.
- Давайте попробуем выполнить более сложный запрос который обычно требует использование JOIN. Перепишем немного наш код (обратите внимание на последнюю строчку student.group.group_name):

main.py > Students

```python
from sqlalchemy import create_engine, Column, Integer, String, ForeignKey
from sqlalchemy.orm import Session, DeclarativeBase, relationship

engine = create_engine("sqlite:///test.db")

class Base(DeclarativeBase): pass
class Groups(Base):
    __tablename__ = "groups"

    group_id = Column(Integer, primary_key=True, index=True)
    group_name = Column(String)
    course = Column(Integer)

class Students(Base):
    __tablename__ = "students"

    student_id = Column(Integer, primary_key=True, index=True)
    last_name = Column(String)
    first_name = Column(String)
    group_id = Column(Integer, ForeignKey("groups.group_id"))
    group = relationship("Groups")

with Session(autoflush=False, bind=engine) as db:
    students = db.query(Students).all()
    for student in students:
        print(f"ID:{student.student_id} ФИ: {student.last_name} {student.first_name} Группа: {student.group.group_name}")
```

Результат:

```
PS C:\Users\Evgen\Desktop\pythonDB> uv run main.py
ID:1 ФИ: Иванов Алексей Группа: ИС-21
ID:2 ФИ: Петрова Мария Группа: ИС-21
ID:3 ФИ: Сидоров Дмитрий Группа: ИС-21
ID:4 ФИ: Кузнецова Анна Группа: ИС-22
ID:5 ФИ: Смирнов Игорь Группа: ИС-22
ID:6 ФИ: Волкова Елена Группа: ИС-22
ID:7 ФИ: Морозов Павел Группа: ПМ-21
ID:8 ФИ: Зайцева Ольга Группа: ПМ-21
ID:9 ФИ: Лебедев Никита Группа: ПМ-21
ID:10 ФИ: Козлова Дарья Группа: ЭК-23
ID:11 ФИ: Новиков Артём Группа: ЭК-23
ID:12 ФИ: Соколова Виктория Группа: ИС-21
PS C:\Users\Evgen\Desktop\pythonDB>
```